

01 API - Gegevens ophalen uit een database

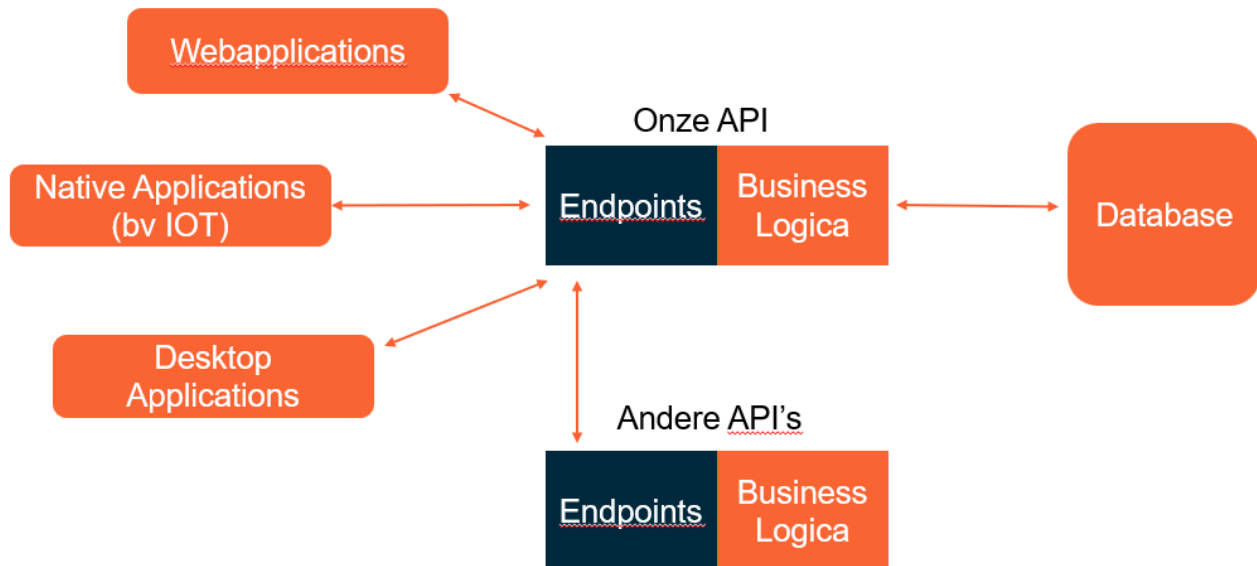
📅 09-09-2026 ⌚ Ongeveer 17 minuten 🗨️ Ongeveer 2615 woorden

In dit hoofdstuk leer je wat een API is en hoe je in Visual Studio een WebAPI maakt. Je leert hoe je met behulp van het Entity Framework een database aanmaakt, code-first, en hoe je de gegevens uit de database haalt. Daarna wordt uitgelegd hoe je de API kunt aanspreken met tools als Postman en Swagger.

Wat is een API?

API staat voor **Application Programming Interface**. Een api is een soort verbidingsbrug tussen twee softwarecomponenten. De API geeft toegang tot functionaliteiten zonder dat een programma alle achterliggende details hoeft te weten. Op die manier kun je als bedrijf bv gegevens delen met andere bedrijven. Bekende voorbeelden van zulke gegevens zijn weersgegevens, afbeeldingen, video's of locatiegegevens. Dat zijn gigantische hoeveelheden data die op verschillende manieren kunnen worden geïnterpreteerd of geanalyseerd. In plaats van zelf software te ontwikkelen voor al die verschillende mogelijkheden, maakt een bedrijf een API (interface) zodat andere bedrijven aan die data kunnen. API's kunnen volledig of deels beveiligd zijn zodat misbruik of vervalsing kan voorkomen worden. Deels beveiligd houdt in dat het opvragen van gegevens dan publiek mogelijk is, maar wijzigingen aanbrengen niet kan zonder toestemming.

Een bedrijf kan dan met die data zelf beslissen wat voor soort applicatie ze ontwikkelen. Dat kunnen bv webapplicaties, desktopapplicaties, IOT-devices of andere API's zijn.



In de praktijk zijn er verschillende soorten API's zoals RestAPI (Representational State Transfer API), SOAP API (Simple Object Access Protocol), GraphQL API (querytaal voor API's die door Facebook zijn ontwikkeld), Streaming API (continue versturen van gegevens)...

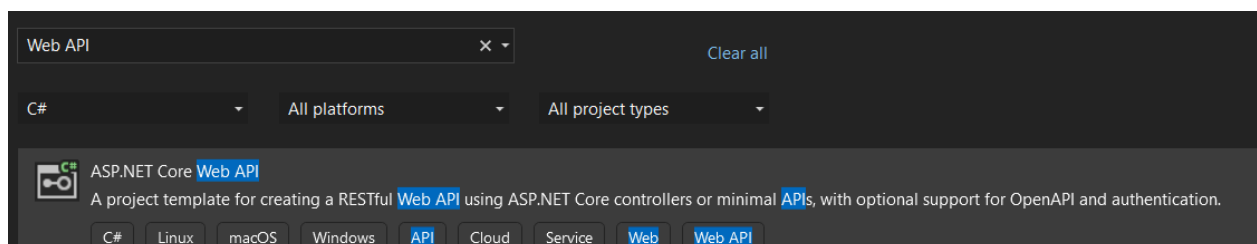
In deze cursus ligt de focus op een RestAPI. Dat is een eenvoudige API die veel gebruikt wordt om informatie te delen via het internet (HTTP). Je stuurt een verzoek naar een web-server en de server stuurt een antwoord, meestal in JSON-formaat. In andere OPO's heb je al geleerd hoe je een fetch (Javascript) of een Http-request (IT-Challenges 2) kunt gebruiken om gegevens op te halen.

Een HTTP-Request is niet alleen gegevens opvragen (GET) maar kan ook toevoegen (POST), bijwerken (PUT) of verwijderen (DELETE) zijn.

WebAPI aanmaken

Visual Studio

Maak in Visual Studio een nieuw project **ASP.NET Core Web API** aan met als naam **WebAPIDemo**. Kies als Framework **.NET 8**.



Terminal

Via de terminal van bv Visual Studio Code kun je ook een nieuw Web API project aanmaken. Visual Studio maakt standaard een Solution aan met een nieuw project. In VSC kun je dus best ook eerst een solution maken, en daarin maak je dan een nieuw project.

```
1  \\ nieuw project aanmaken                                pwsh
2  dotnet new sln -n WebAPIDemo
3  \\ nieuw .NET 8.0 project aanmaken dat gebruik maakt van Controllers
4  dotnet new webapi -n WebAPIDemo --framework net8.0 --use-controllers
5  \\ project toewijzen aan de solution
6  dotnet sln WebAPIDemo.sln add WebAPIDemo/WebAPIDemo.csproj
```

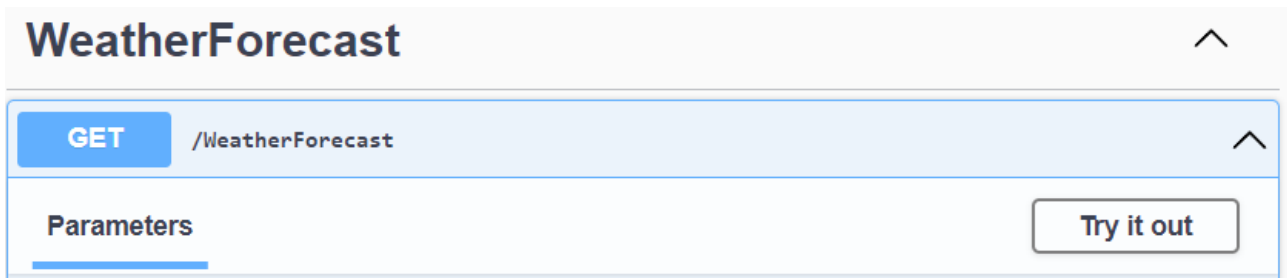
Demoproject

Een nieuw project bevat een demo van een WeerAPI. Hierin zijn volgende belangrijke documenten aanwezig:

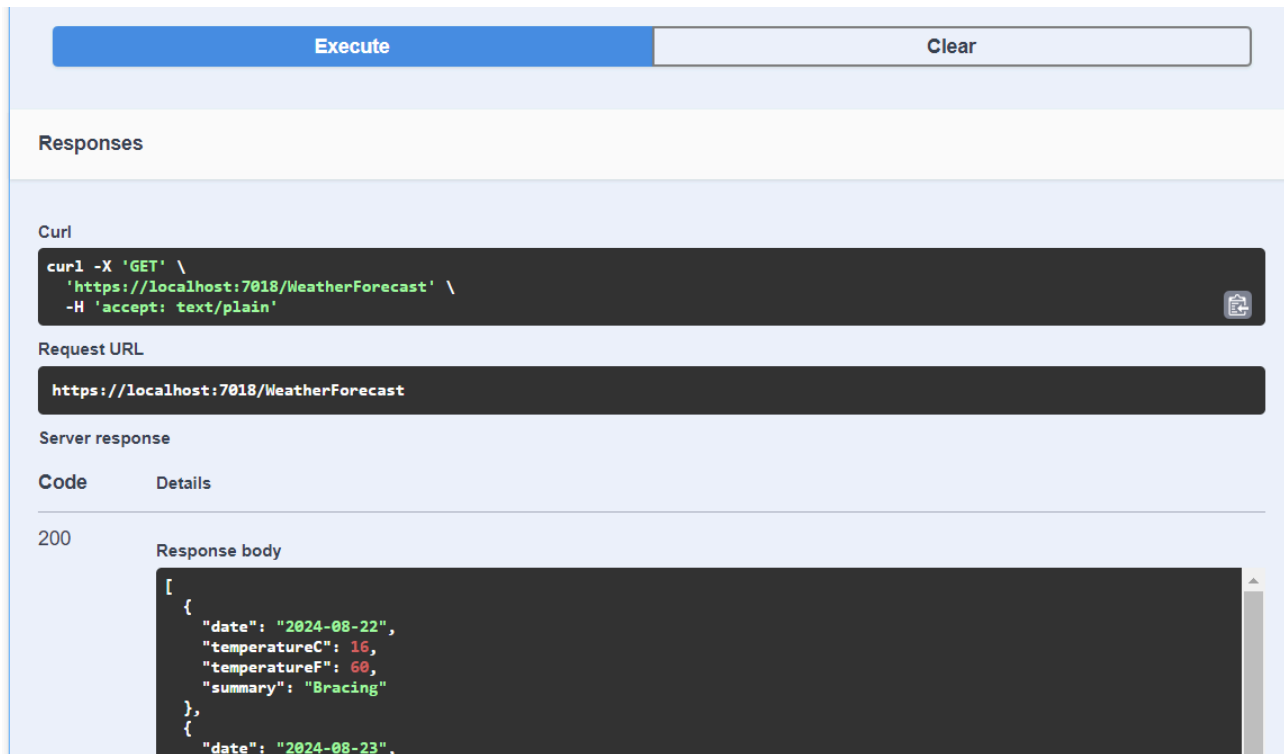
- **Program.cs:** Dit is het startpunt van de applicatie. Hier vind je ook terug welke services worden geladen.
- **appsettings.json:** Hier komt de databaseverbinding.
- **Properties\launchSettings.json:** Hier vind je terug op welke URL's je de webserver kunt opstarten.
- **Controllers\WeatherForecastController:** Dit is de controller voor het WeatherForecast - endpoint. Een controller bevat de logica die wordt uitgevoerd als je naar localhost: *poortnr*\weatherforecast surft. Je kunt dit vergelijken met de logica van het ViewModel in MVVM. Standaard wordt een GET-Request uitgevoerd en wordt de bijhorende code uitgevoerd.

Als je het project start, kom je op een pagina van Swagger terecht. Dat is een plugin om de API-request uit te voeren.

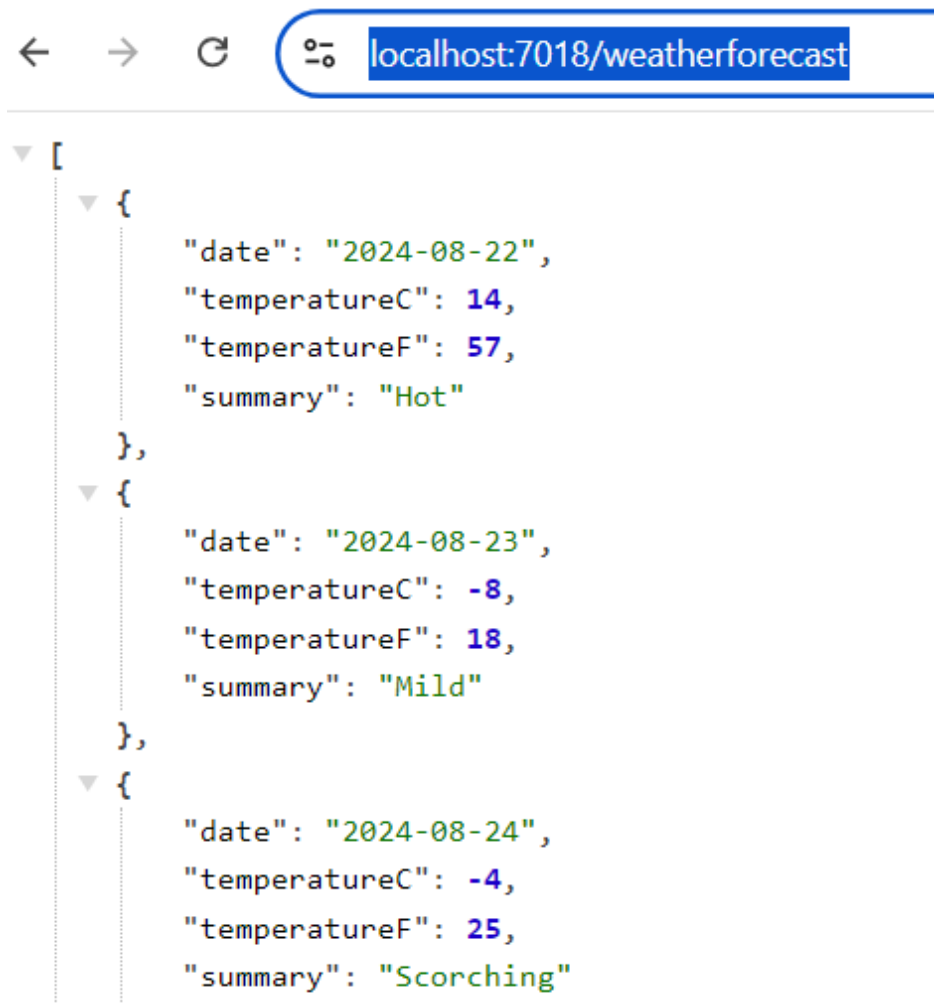
- Klik rechts op *Try it out*
- Klik op *Execute*



Je krijgt een code 200 (succes) en de resultaten in JSON-formaat.



Je kunt ook rechtstreeks naar een end-point surfen.



Het resultaat van de pagina is nu enkel een lijst van gegevens in JSON-formaat.

Als je het project start, zul je een command-venster terugvinden in je taakbalk. Achterliggend is een webserver opgestart die luistert naar inkomende verzoeken. C# kan niet rechtstreeks gecompileerd worden door een browser. Daarom heb je aan de serverkant een webserver nodig die de backend-logica uitvoert. In het commandvenster kun je informatie van deze webserver terugvinden.

Logger

Je kunt, tijdens het uitvoeren van je applicatie, boodschappen sturen naar het command-venster. Daarvoor maak je gebruik van een ingebouwde logger. In het voorbeeld wordt eerst de logger geïnjecteerd in de controller. Daarna kun je deze logger aanspreken en boodschappen sturen (regel 11).

```
1 private readonly ILogger<WeatherForecastController> _logger;
2
3 public WeatherForecastController(ILogger<WeatherForecastController>
4 logger)
5 {
6     _logger = logger;
7 }
8
9 [HttpGet(Name = "GetWeatherForecast")]
10 public IEnumerable<WeatherForecast> Get()
11 {
12     _logger.LogInformation("De Index-pagina is geladen.");
13     return Enumerable.Range(1, 5).Select(index => new WeatherForecast
14     {
15         Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
16         TemperatureC = Random.Shared.Next(-20, 55),
17         Summary = Summaries[Random.Shared.Next(Summaries.Length)]
18     })
19     .ToArray();
20 }
```

Er zijn verschillende niveaus van logging die je kunt gebruiken om de ernst van het bericht aan te geven:

- **LogTrace**: Gedetailleerde log, voornamelijk voor debugging.
- **LogDebug**: Informatie voor debugging.
- **LogInformation**: Informatie over normale werking.
- **LogWarning**: Warnings of mogelijke problemen.
- **LogError**: Fouten die zijn opgetreden.
- **LogCritical**: Kritieke fouten die onmiddellijke aandacht vereisen.

```
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\u0157634\OneDrive - Thom
info: WebAPIDemo.Controllers.WeatherForecastController[0]
      De Index-pagina is geladen.
```

Opkuisen project

Verwijder `WeatherForecast.cs` en `WeatherForecastController.cs` uit het project.

Entity Framework

Nu je weet hoe je een demo-project kunt aanmaken, opstarten en testen is het tijd om zelf een API te maken met gegevens uit een SQL-database. Om de databaseverbinding te maken, gebruik je in deze cursus **Entity Framework** in plaats van Dapper. Entity Framework is een ORM (Object Relational Mapping) framework dat de C#-modellen omzet naar SQL-tabellen. vb je project gebruikt *string* voor een tekst-variabele en die moet omgezet worden naar *varchar*.

Met Entity Framework kun je de database en de tabellen aanmaken vanuit je code. Dit principe noemt **Code-First** en heeft als voordeel dat je geen ingewikkelde SQL-scripts hoeft te schrijven.

De werkwijze is als volgt: Je vertrekt vanuit je model, maakt een migratie aan en update je database. Als je nadien wijzigingen aanbrengt in je model, maak je een nieuwe migratie aan en update je opnieuw de database.

Voor je kunt beginnen met een database aan te maken, moet je nog wel eerst Entity Framework instellen. Het instellen van Entity Framework doe je in 4 stappen:

- Packages installeren
- Databaseverbinding instellen in `appsettings.json`
- DbContext aanmaken
- DbContext registeren voor Dependency Injection in `Program.cs`

Packages installeren

Visual Studio

- Ga naar Tools - NuGet Package Manager - Manage NuGet Packages for Solution
- Installeer Microsoft.EntityFrameworkCore (Core)
- Installeer Microsoft.EntityFrameworkCore.SqlServer (specifiek voor SQL-database)
- Installeer Microsoft.EntityFrameworkCore.Tools (om migraties uit te kunnen voeren)

Terminal

```
1 # Installeer Entity Framework Core (basis) pwsh
2 dotnet add package Microsoft.EntityFrameworkCore --version 8.*
3
4 # Installeer de SQL Server-provider
5 dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.*
6
7 # Installeer de tools voor migraties en scaffolding
8 dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.*
```

Databaseverbinding instellen in appsettings.json

In `appsettings.json` maak je een databaseverbinding aan. Controleer dit ook steeds in latere projecten! Dit is afhankelijk van waar je de database wilt hosten en hoe de database noemt. Dit kan lokaal zijn of online zoals bv Azure.

Docker

in de opleiding gebruik je de SQL-server van Docker. Je geeft de naam op van de server (eventueel met poortnr), de naam van de database, gebruikersnaam, wachtwoord en Voeg onderstaande regels toe aan `appsettings.json` :

```
1 "ConnectionStrings": { json
2   "LocalDBConnection": "Server=localhost,1433;Database=WebAPIDemo;User
3   Id=sa;Password=Pass.Word;TrustServerCertificate=true"
4 },
```

Visual Studio

Standaard wordt er door Visual Studio ook een lokale server geïnstalleerd. Je geeft de naam op van de server, de naam van de database en eventuele opties. Integrated Security=True (of Trusted_Connection=True) geeft aan dat Windows-authenticatie moet worden gebruikt voor het verbinden met de database in plaats van SQL Server-authenticatie. Voeg onderstaande regels toe aan `appsettings.json` :

```
1      "ConnectionStrings": {  
2          "LocalDBConnection": "Server=  
3      (localdb)\\mssqllocaldb;Database=WebAPIDemo;Integrated Security=True"  
        },  
    },  
}
```

json

DbContext aanmaken

Een `DbContext` is een klasse die alle informatie bevat over welke modellen gebruikt moeten worden in de database en onder welke naam.

Maak eerst een model `Product` aan, in een map `Models` met volgende properties:

- `Id` (int), `Naam` (string) en `Prijs` (decimal)

```
1      public class Product  
2      {  
3          public int Id { get; set; }  
4          public string Naam { get; set; }  
5          public decimal Prijs { get; set; }  
6      }
```

CS

Maak daarna een klasse `WebAPIDemoContext.cs` aan, in een map `Data`.

```
1      using Microsoft.EntityFrameworkCore;  
2      using WebAPIDemo.Models;  
3  
4      namespace WebAPIDemo.Data  
5      {  
6          public class WebAPIDemoContext: DbContext  
7          {  
8              public WebAPIDemoContext(DbContextOptions<WebAPIDemoContext>  
9              options) : base(options) { }  
10  
11              public DbSet<Product> Producten { get; set; }  
12  
13              protected override void OnModelCreating(ModelBuilder modelBuilder)  
14              {  
15                  base.OnModelCreating(modelBuilder);  
16              }
```

CS

```

17         modelBuilder.Entity<Product>
18         ().ToTable("Product").Property(p=>p.Prijs).HasColumnType("decimal(18,2)");
        }
    }
}

```

In deze klasse plaats je bovenaan verwijzingen naar de map `Models` en het `Microsoft.EntityFrameworkCore`. Je kunt, net zoals in *Programming Advanced*, ook alle verwijzingen in een `GlobalUsings.cs` plaatsen.

`WebAPIDemoContext.cs` erft over van `DbContext` en in de constructor bepaal je dat er opties kunnen worden meegegeven. Dit zijn opties zoals de connectiestring, de databaseprovider... In de volgende stap registreer je deze klasse in `Program.cs` en geef je de juiste connectiestring mee.

`DbSet<Product> Producten { get; set; }`: De collectie *Producten* wordt gemapped naar de tabel *Product* uit de database.

De methode **`OnModelCreating`** wordt overschreven zodat je zelf kunt bepalen hoe tabellen worden aangemaakt in de database. Van het model *Product* wordt er een tabel *Product* gemaakt. Je kunt hier zelf kiezen welke naam er wordt gebruikt. De eigenschap *Prijs* wordt ingesteld zodat deze een decimale notatie heeft van 18 cijfers met 2 decimalen.

DatabaseContext registreren voor Dependency Injection in `Program.cs`

Als laatste stap registreer je `WebApiDemoContext` als een service in `Program.cs`. Hier bepaal je, als optie, welke databaseconnectie er wordt gebruikt voor je databasecontext.

```

1     builder.Services.AddDbContext<WebAPIDemoContext>(options =>
        options.UseSqlServer(builder.Configuration.GetConnectionString("LocalDBConne

```

Migraties

Als Entity Framework goed geconfigureerd is, kun je de database aanmaken/updaten. Eerst maak je een migratie aan. Dit is een bestand dat wijzigingen in je models ten opzichte van je database registreert. Elke keer je een migratie aanmaakt, wordt er ook een snapshot gemaakt van hoe je volledige database er moet uitzien. Met *Update-database* voer je de migraties ook effectief uit en wordt de database geüpdatet of aangemaakt als die nog niet bestaat.

Visual Studio

Ga naar View - Other Windows - Package Manager Console Typ: Add-Migration
"Initial_create" Typ: Update-database

Terminal

```
1  \\ migratie aanmaken                                pwsh
2  dotnet ef migrations add Initial_create
3  \\ database updaten
4  dotnet ef database update
```

Opmerkingen:

- Geef steeds een zinvolle naam aan je migraties.
- Werk altijd via Migraties en pas nooit rechtstreeks iets aan in je database! Dit geeft conflicten in de volgende migratie.

Als je models toevoegt, moet je ook elke keer de databasecontext aanpassen en een nieuwe migratie uitvoeren.

Database bekijken

Om te controleren of je database aangemaakt is en om bv testgegevens in te geven, kun je gebruik maken van:

- SQL Server Object Explorer (in Visual Studio)
- Visual Studio Code (met SQL Server extensie)
- Azure Data Studio
- ...

API Controller

De gegevens in de database kun je opvragen en beschikbaar stellen via een API-controller. Dit is een klasse waarin je het type verzoek opgeeft en wat er moet gebeuren als je zo'n verzoek binnenkrijgt op je webserver. Je kunt ook opgeven op welke URL of endpoint dit verzoek kan binnenkomen.

Maak, in de map Controllers, een nieuwe API-controller *ProductController*.

```
1  using Microsoft.AspNetCore.Http;
2  using Microsoft.AspNetCore.Mvc;
3
4  namespace WebAPIDemo.Controllers
5  {
6      [Route("api/[controller]")]
7      [ApiController]
8      public class ProductController : ControllerBase
9      {
10     }
11 }
```

Zoals je kunt zien wordt er een route gedefinieerd. Als je deze controller wil aanspreken, surf je naar `localhost:poortnr/api/Product`. De `ApiController` erft over van `ControllerBase`.

Dependency Injection

Je hebt de `databasecontext` aangemaakt als een service in `Program.cs`. Je kunt de service injecteren in je controller op de volgende manier:

```
1  private WebAPIDemoContext _context;
2
3  public ProductController(WebAPIDemoContext context)
4  {
5      _context = context;
6  }
```

Elke keer als je nu `_context` aanspreekt, kun je de `DbSet Producten` aanspreken en kun je gegevens opvragen, toevoegen, wijzigen en verwijderen.

Meerdere gegevens opvragen

Wil je alle gegevens ophalen, maak je een methode aan met het kenmerk `[HttpGet]`. De naam van de methode is minder belangrijk. Als je een GET-verzoek verstuurt, kom je automatisch bij deze methode terecht, ongeacht wat de naam is.

LINQ

```
1 [HttpGet]
2 public List<Product> GetAll()
3 {
4     return _context.Producten.ToList();
5 }
```

SQL

```
1 [HttpGet]
2 public List<Product> GetAll()
3 {
4     return _context.Producten
5         .FromSqlRaw("SELECT * FROM Producten")
6         .ToList();
7 }
```

Endpoint: localhost:poortnr/api/Product/

Wil je graag je endpoint benoemen, kun je de naam toevoegen aan het kenmerk.

```
1 [HttpGet("GetAll")]
2 public List<Product> GetAll()
3 {
4     return _context.Producten.ToList();
5 }
```

Endpoint: localhost:poortnr/api/Product/GetAll

Als je de applicatie start, kun je met Swagger testen of je gegevens worden opgehaald.

GET	/api/Product/GetAll
Parameters	
No parameters	
Execute	

Één item ophalen

Wil je één item ophalen, heb je een argument nodig dat je meegeeft als je het endpoint aanspreekt.

LINQ

```
1 [HttpGet("{id}")]
2 public Product GetProduct(int id)
3 {
4     return _context.Producten.FirstOrDefault(x => x.Id == id);
5 }
```

CS

SQL

```
1 [HttpGet("{id}")]
2 public Product GetProduct(int id)
3 {
4     return _context.Producten
5         .FromSqlRaw("SELECT TOP(1) * FROM Producten WHERE Id = {0}", id)
6         .AsEnumerable()
7         .FirstOrDefault();
8 }
```

CS

Endpoint: localhost:poortnr/api/Product/1

ActionResult

Met voorgaande methoden is er een probleem. Je kunt alleen maar resultaat teruggeven van het type `List<Product>` of `Product`. Als je echter geen resultaten hebt of als je een verkeerde input krijgt, wil je graag een gepaste foutmelding geven. Daarvoor gebruik je `ActionResult` in de declaratie van je methode. Je kunt dan ook een statuscode meegeven door middel van `OK`, `NotFound` of `BadRequest`.

```
1 [HttpGet]
2 public ActionResult<List<Product>> GetAll()
3 {
4     var producten = _context.Producten.ToList();
5     if (producten.Count > 0) {
6         return Ok(producten);
7     }
8     else
9     {
10         return NotFound("Er zijn geen resultaten gevonden");
11     }
12 }
```

CS

en

```
1 [HttpGet("{id}")]
2 public ActionResult<Product> GetProduct(int id)
3 {
4     Product product = _context.Producten.FirstOrDefault(x => x.Id == id);
5     if (product == null)
6     {
7         return NotFound($"Product {id} kan niet worden gevonden in de
8 database");
9     }
10    return Ok(product);
11 }
```

CS

Async - Await

Databaseoperaties, zoals het ophalen van grote hoeveelheden gegevens, kunnen soms veel tijd in beslag nemen. In de bovenstaande gevallen wordt de code synchroon uitgevoerd. Dit wil zeggen dat elke regel code pas uitgevoerd wordt als de vorige regel code klaar is. In programmeertaal wordt wel eens de term `thread` gebruikt. Een `thread` kun je beschouwen als een wachtrij aan de kassa. Je moet wachten tot je aan de beurt bent en elke klant wordt één voor één afgewerkt. Sommige applicaties kunnen `multi-threading` wat betekent dat meerdere regels code parallel naast elkaar kunnen draaien. Net zoals je meerdere kassa's hebt in de winkel. Maar aan elke kassa of `thread` moet je nog steeds je beurt afwachten.

Om te voorkomen dat je met een grote databaseoperatie de `thread` zou ophouden, kun je `async` en `await` toepassen op je methode. Dat is zoals je aan de kassa staat met een volle kar. Terwijl je alles op de band aan het leggen bent, kun je iemand anders snel laten voorsteken.

Technisch uitgelegd: **Threads in een Web API** In een Web API krijgt elk inkomend HTTP-verzoek meestal een thread toegewezen.

- Als een request langdurig een database oproept zonder `async`:
- Die thread is bezet totdat de query klaar is.
- Andere requests moeten wachten tot er een vrije thread is. **Met `async/await`:**
- De thread wordt vrijgegeven terwijl de databasequery loopt.
- Andere requests kunnen die thread gebruiken.
- Zodra de query klaar is, gaat de verwerking verder op een beschikbare thread.

In de code pas je dat als volgt aan:

- Voeg `async Task<>` toe aan `ActionResult`.
- Voeg `await` toe aan je databaseoperatie en gebruik een Async-methode zoals `ToListAsync()` of `FirstOrDefaultAsync()`.

```
1  [HttpGet]
2  public async Task<ActionResult<List<Product>>> GetAll()
3  {
4      var producten = await _context.Producten.ToListAsync();
5      if (producten.Count > 0) {
6          return Ok(producten);
7      }
8      else
9      {
10         return NotFound("Er zijn geen resultaten gevonden");
11     }
12 }
```

CS

Figuur 1: Uitleg asynchroon programmeren

Het voorbeeld uit het filmpje kun je terug vinden op volgende link: [Github - voorbeeld th-reading](#)

[Hier vind je het WebApiDemo-project.](#)

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM